Project-Management Pipeline Review

Oct 22, 2026 · @Claudius Gallus

Summary

The pipeline's design is sound and its trust discipline is the right one for a cheap-worker fleet, but reading the current definitions shows Opus tokens spent on work that is deterministic or duplicated. No run of the current versions has been instrumented, so every cost claim here is structural, not measured. Three Opus stages at high effort surround each Sonnet worker; the supervisor copies every step's text into the worker prompt and performs about ten git actions per step as separate model turns; the decomposer may embed verbatim code that Opus writes and Sonnet retypes. Opus 5.5 costs 2× Sonnet 5.5 per token, so the cheap-worker tier saves less than the design assumes; the real levers are fewer Opus tokens per step and Haiku for verbatim steps.

Top changes, in order of expected return:

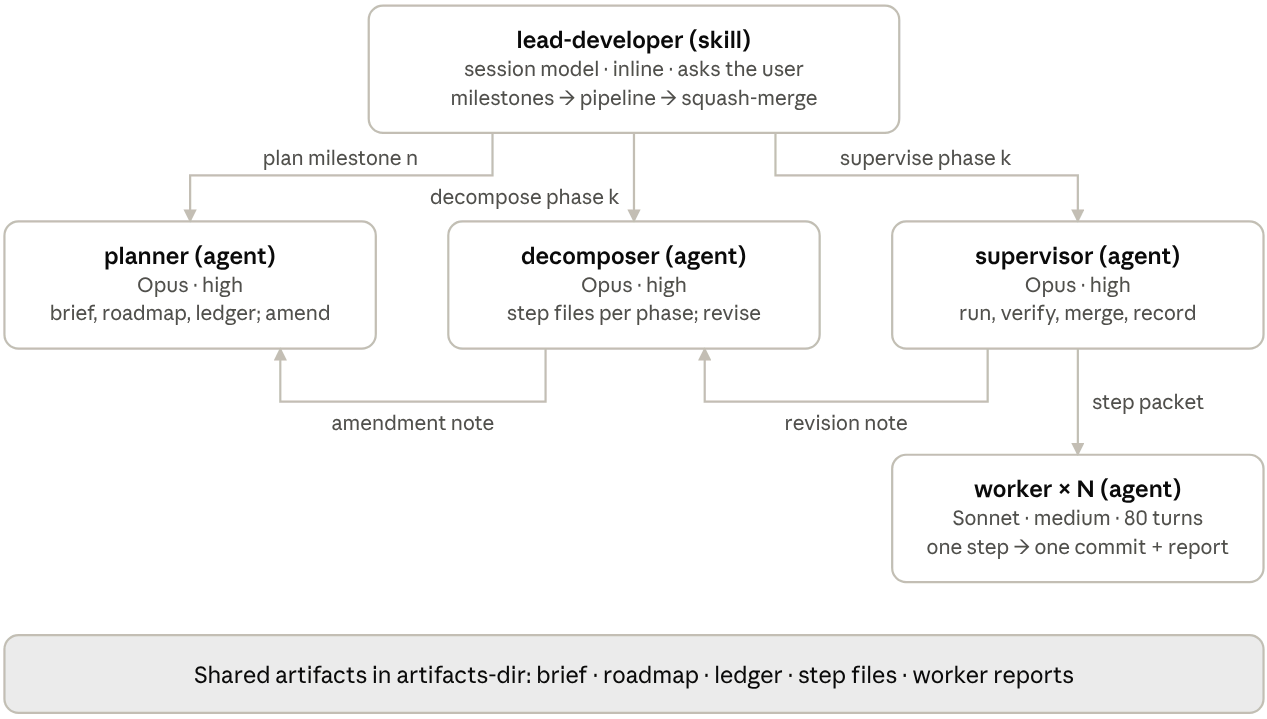
1. Script the supervisor's mechanics (worktree create and base gate, acceptance and scope verify, merge and ledger record). Cuts Opus turns per step from about ten to about four and makes a Sonnet supervisor testable.
2. Let workers read their own step file and stop the supervisor re-emitting packets. Removes the whole step text from the supervisor's output; for a step carrying embedded code that is most of the file.
3. Instrument the ledger with per-step tokens, tool uses, wall-clock and retries, then run the ablations in the cost section before changing any model or effort pin.
4. Flatten the amendment path: the decomposer returns `needs-amendment` instead of spawning the planner. Drops a nesting level and the decomposer's `Agent` tool.
5. Drop the `rollback` field and the default gate step; both spend decomposer and worker tokens on checks the supervisor repeats anyway.

None of these weakens the never-trust discipline. Each removes model tokens from deterministic work or duplicated text.

System overview

Five stages cooperate only through Markdown files keyed by a plan name; four are agent definitions and one is a skill. The lead developer is a skill because it must interview the user and because running inline keeps the deepest chain (supervisor → decomposer → planner) inside the harness's nesting limit.

Every stage above the worker runs on Opus at high effort



pipeline stages · 5 stages, 3 loops

Control flows down. The loops back up carry revision notes (supervisor → decomposer) and amendment notes (decomposer or supervisor → planner); the supervisor's amendment path is not drawn. Stages share nothing but the artifact files in the band.

Stage	Kind	Definition (words)	Owns	Tools
lead-developer	skill, inline at depth 0	2,341	project plan, project ledger, milestone branches, squash-merge	the session's
planner	agent	2,851	brief, roadmap, ledger seed; amend operation	Read, Write, Edit, Grep, Glob, Bash

Stage	Kind	Definition (words)	Owns	Tools
decomposer	agent	3,139	step files, ledger step registry; revise operation	same + Agent
supervisor	agent	3,613	worktrees, verification, merges, ledger results	same + Agent
worker	agent, 80-turn cap	1,253	one step: one commit and one report	no Agent

A "Shared model" section of about 1,200 words is duplicated verbatim in the planner, decomposer and supervisor and must be kept in sync by hand. Every definition is loaded in full on every invocation, including each worker launch and each sibling spawn.

Effectiveness

The definitions describe a pipeline that produces verified, merged work; the failure modes that remain are protocol gaps and assumptions about the harness, not defects in the core loop. Every claim in this section comes from reading the five definitions and the Claude Code documentation (checked 2026-10-22); no instrumented run of the current versions exists.

What the design gets right, by construction:

- Ground-truth verification: the supervisor re-runs every acceptance command and diffs the worktree against BASE, so a worker's report can be wrong without anything wrong merging; the "check the check" rule also covers content manufactured to satisfy a grep.
- Projection into self-contained step packets: a worker that sees only its packet cannot wander, and a missing fact surfaces as a reported step defect rather than as exploration.
- Base assertion with "stop, never repair": the first action of a dependent step detects a mis-based worktree behind the supervisor's own gate, and the worker is forbidden to fetch, merge or rebase.
- Disjoint `files_in_scope` with one worktree per parallel step at an exact BASE: merges cannot conflict unless the decomposition is wrong, and a conflict is routed as a decomposition bug.

- Committed ledger state plus the revision and amendment loops: the run is resumable after a crash, and a plan defect reaches the artifact's owner instead of being patched where it was found.

Where it can still fail:

#	Where	Finding	Consequence	Severity
E1	CLAUDE.md, all agents	Harness facts are asserted in prose and never checked at run time. Checked against the Claude Code docs on 2026-10-22: nesting is three layers below the main conversation by default and concurrency 20, both set by environment variables; `isolation: worktree` bases on the remote default branch unless `worktree.baseRef` is `head`. Undocumented: whether foreground launches in one message run concurrently.	An environment variable, a settings change or a Claude Code release changes one and the pipeline fails or serializes mid-run with no diagnostic.	Medium
E2	decomposer, supervisor	The amendment loop from a supervisor-spawned decomposer reaches the third layer below the main conversation (supervisor 1, decomposer 2, planner 3), exactly the documented default limit; `CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH` can lower it.	Works at the default; an environment with a lower depth, or a person invoking the supervisor from inside another agent, loses amendments mid-run.	Medium
E3	supervisor	A `judgment` step is resolved by the supervisor itself, but nothing says how its own work is scoped, checked, committed or recorded.	Unverified, unrecorded commits on the working branch from the one stage that is supposed to verify everything.	Medium
E4	lead-developer, Merge step	Step files and worker reports are deleted before the squash; the ledger that is kept still points at them.	After every milestone the per-step evidence (what was asked, what was done, what the acceptance printed) is gone, so neither a post-mortem nor a cost attribution can be done later.	Medium

#	Where	Finding	Consequence	Severity
E5	worker	Subagents load every CLAUDE.md level the main conversation loads, including the user's global one (documented), whose shell rules (no `cd <dir> &&`, PowerShell first) contradict the worker's path discipline.	A Sonnet worker spends turns reconciling two instruction sets; `omitClaudeMd: true` in the worker frontmatter is the documented opt-out.	Medium
E6	decomposer, step schema	`rollback` is authored per step, but a worktree step is rolled back by discarding the worktree and an in-tree step by `git reset --hard <base>`; no stage runs the field's text.	Decomposer output nobody acts on; a wrong rollback would be trusted if anyone did.	Low
E7	decomposer, supervisor	The gate step re-runs the phase DoD that the supervisor must re-run anyway, and the supervisor's own rule says a gate report is a lead it may not rely on.	One worker launch, worktree, merge and ledger commit per phase for a result the definition forbids trusting; a failing gate adds a revision cycle before the supervisor's own run finds the same failure.	Low
E8	worker, report schema	`acceptance` carries each command's verbatim output with no cap.	A full test-suite log lands in a committed report and in the supervisor's context.	Low
E9	README	Worker listed as "Sonnet / low"; the definition pins `medium`.	Wrong cost expectation for anyone reading the README.	Low
E10	supervisor, Decide	Retry (a) says to "hand a corrected packet" for a worker-level miss, while correcting a packet is defined as option (b) with a Revisions row and a committed step file.	The two options blur; a corrected packet can run under (a) with no artifact trail.	Low

Efficiency

Most avoidable spend is Opus output and Opus turns on deterministic or duplicated work: packet re-emission, git bookkeeping, verbatim code in step files, and a gate step whose checks the supervisor repeats. Nothing has been measured for the current definitions; the table ranks the sinks by reading the protocols, and the cost section says how to measure them.

Prices that set the ratios (Anthropic API, cached 2026-10-06, $ per million tokens):

Model	Input	Output	Cache read
Fable 5.1 (likely the lead-developer's session model)	10.00	50.00	0.25
Opus 5.5 (planner, decomposer, supervisor)	4.00	20.00	0.20
Sonnet 5.5 (worker)	2.00	10.00	0.20
Haiku 5.5	0.10	0.50	0.01

Opus is only 2× Sonnet per token, so routing typing to Sonnet saves at most half of that work's cost; Haiku is 20× cheaper than Sonnet. Effort is the other first-order lever: at high effort every Opus turn carries thinking tokens billed as output.

#	Where	What is spent	Why it is avoidable	Lever
F1	supervisor, launch	The packet template copies `objective`, `context`, `actions`, `files_in_scope` and `acceptance` from the step file into the worker prompt, so every step's text is written once by the decomposer and again by the supervisor, at Opus output price plus the thinking around it.	The step file is already on disk in the worktree; only the "files changed in step X" references need resolving, and the decomposer knows those paths when it writes the step.	Workers read their own step file; decomposer resolves references at write time.

#	Where	What is spent	Why it is avoidable	Lever
F2	supervisor, per step	The protocol mandates about ten git actions per step as separate model turns (worktree add, rev-parse and merge-base gates, launch, acceptance re-run, scope diff, merge, ledger edit, ledger commit, worktree and branch removal), each re-reading a growing context and thinking at high effort.	Every one of those turns is deterministic given the step file and the ledger.	Shell scripts for create-gate, verify, merge-record, cleanup; the supervisor calls four commands per step and reasons only about the verdict.
F3	decomposer, `actions`	The Specification level paragraph permits verbatim code where a variant would be silently wrong and warns that writing every step verbatim makes decomposition the dominant cost, but nothing enforces the spec-level default.	Verbatim code means Opus writes it, Sonnet retypes it and Opus verifies it; at 2× price that is dearer than Opus implementing directly.	Spec-level `actions` as the rule; verbatim only for sign conventions, formulas and delicate contracts, named in the step; measure revision rate.
F4	all three agents	A 1,200-word Shared model in every planner, decomposer and supervisor context; the planner never uses the worker report protocol or step fields, the decomposer never uses worktree mechanics.	Stable prefix, so mostly cache reads in dollars; the cost is reasoning noise and a three-file manual sync on every edit.	Keep a 40-line core in all three; move stage-specific protocol to the stages that act on it; add a drift check script.

#	Where	What is spent	Why it is avoidable	Lever
F5	decomposer, supervisor	Gate step: one worker, worktree, merge and ledger commit per phase to run DoD commands the supervisor re-runs.	By the supervisor's own rule the gate report is a lead, never a substitute, so the step carries no information the phase check does not.	Drop the default gate step; keep it only for DoD items too slow to run twice.
F6	step schema	`rollback` written for every step.	No stage runs it; worktrees are discarded, in-tree steps reset to base.	Remove the field (major schema bump).
F7	lead-developer, planner	Each surveys the repo's formatter and lint configuration against a five-tool table before writing anything.	Deterministic file detection done by an Opus agent reading config files.	One script that prints the gates found and the ignore entry to add.
F8	lead-developer	Every agent return, every milestone DoD command output and every git check accumulates in the session context at the session model's price, across all milestones.	Only the verdict matters; raw DoD output is noise after the check passes.	DoD runner script that prints one PASS or FAIL line per item; tail long outputs.
F9	worker	Sonnet at medium for every mechanical step, including ones that apply verbatim content in a handful of turns.	Haiku is 20× cheaper per token and the supervisor catches its failures either way.	A `worker-lite` definition on Haiku for verbatim-apply steps, chosen by a step `tier` field; measure before adopting.

#	Where	What is spent	Why it is avoidable	Lever
F10	supervisor, decomposer	A clarification `needs-human` re-spawns the whole stage, which re-reads the repo and artifacts from scratch.	Unavoidable with stateless subagents; the lead-developer interview keeps it rare.	Accept; count occurrences.
F11	supervisor	Parallelism depends on the decomposer's graph flatness, on per-worktree toolchain bootstrap, and on foreground launches in one message running concurrently, which the docs do not state.	Never measured.	Record wave count and per-step wall-clock in the ledger (see Cost evaluation).

Implementation improvements

These are file-level edits that stand on their own; the version column follows the repo's rule (patch = wording, minor = capability or pins, major = packet, schema or `RESULT:` contract). Edits to the Shared model section apply to all three agents in one commit.

#	File	Edit	Fixes	Bump
I1	`supervisor/AGENT.md` §3	Packet becomes `worktree`, `plan-name`, `id`, `artifacts-dir`, `step-file` (path inside the worktree). Drop the per-field copy.	F1	major
I2	`worker/AGENT.md`	Read `step-file` first; act on its `objective`, `context`, `actions`, `files_in_scope`, `acceptance`; ignore `route`, `depends_on`. Packet table shrinks to five rows.	F1	major

#	File	Edit	Fixes	Bump
I3	`decomposer/AGENT.md`, step schema	Replace "files changed in step X" references with the concrete paths (the decomposer wrote that step's `files_in_scope`); keep the symbolic form only for a revision that cannot know them. Remove `rollback`. Add `tier: standard | lite` under `route`.	F1, F6, F9	major
I4	`decomposer/AGENT.md`, step schema	`acceptance` becomes two fenced blocks, `acceptance_cmd` (sh) and `acceptance_expect` (exact stdout, or `exit 0`), so a script can run and compare it.	F2	major
I5	`decomposer/AGENT.md`, Gate steps	Make gate steps opt-in: only when a DoD item takes longer than about a minute to run. Default phases end with the supervisor's own DoD run.	E7, F5	minor
I6	`decomposer/AGENT.md`	Remove `Agent` from `tools`; replace "spawn the planner with an amendment note" with `RESULT: needs-amendment` followed by the note.	E2	major
I7	`supervisor/AGENT.md` §1, §4e	On a decomposer `needs-amendment`, the supervisor spawns the planner, then re-spawns the decomposer. Add the same handling to `lead-developer` for the decompose step.	E2	major

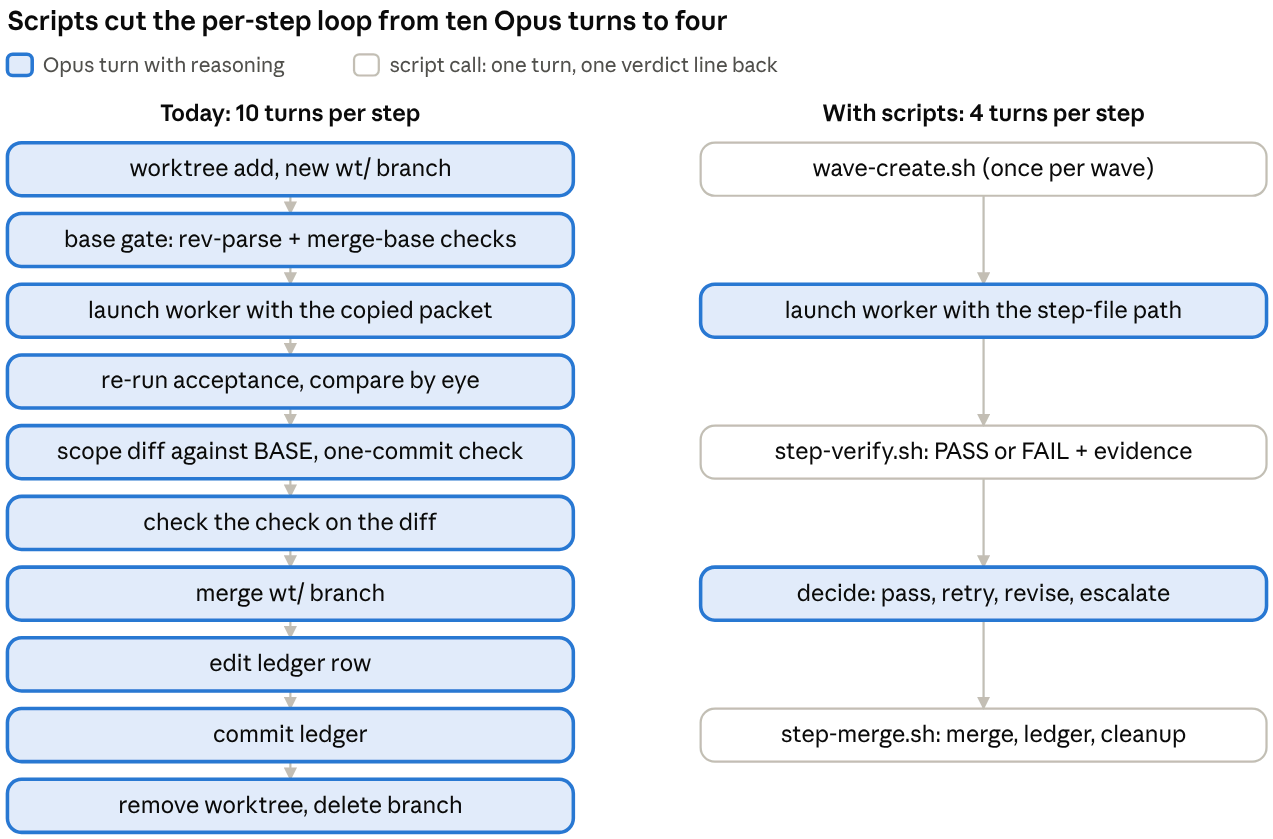
#	File	Edit	Fixes	Bump
I8	`supervisor/AGENT.md`, Routing	Define the supervisor's own handling of a `judgment` step: same scope and acceptance checks on its own diff, commit `supervise(<plan-name>): step <id> (judgment)`, ledger row like any step.	E3	minor
I9	`lead-developer/SKILL.md`, Merge step	Before deleting step files and reports, `git tag archive/<project-name>-m<n> <milestone-branch-HEAD>`; keep the tag out of the squash. The post-mortem data survives at zero cost.	E4	minor
I10	`worker/AGENT.md` §5	`acceptance` in the report carries the command, the exit code and the last 40 lines of output, or the lines matching the expected string.	E8	minor
I11	`worker/AGENT.md`, Path discipline	Add `omitClaudeMd: true` to the frontmatter (the documented opt-out). Keep `git -C`; prefer a tool's own directory flag over `cd <worktree> &&` where one exists.	E5	minor
I12	Shared model, all three	Keep Artifacts, Calling a sibling, Return protocol, Artifact style (about 40 lines). Move Worker report protocol to decomposer and supervisor only; Integration model to planner (ledger fields) and supervisor; Step fields to supervisor.	F4	minor

#	File	Edit	Fixes	Bump
I13	`project-management/scripts/` (new)	`check-shared-model.sh`: extracts the Shared model section from the three AGENT.md files and exits 1 on any difference. Run it from `skill-version-check` or a pre-commit hook.	F4	n/a
I14	`lead-developer/SKILL.md` §5, `planner/AGENT.md` §3	Replace the five-tool survey table with a call to `find-formatter-gates.sh <artifacts-dir> <plan-name>`, which prints each gate found and the exact ignore entry; the agent applies what it prints.	F7	minor
I15	`README.md`	Worker line: Sonnet / medium.	E9	n/a
I16	`project-management/CLAUDE.md`	Spawn table: decomposer spawns nothing; supervisor spawns worker, decomposer, planner. Add: every harness fact in these files (nesting, concurrency, worktree base) is re-checked by the smoke test after a Claude Code upgrade.	E1, E2	n/a

All version bumps and their `manifest.json` entries land in the same commit as the edit.

Architecture improvements

The structural change is to move every deterministic action out of the model and into scripts, then let the models pull their inputs from disk instead of having an Opus stage push them. Everything else follows from those two moves.



supervisor per-step loop · before and after scripts

A model turn remains only where a decision is made; wave creation runs once per wave, not per step. Each script prints one verdict line, so the supervisor's context stops accumulating raw git output.

1. **Pipeline scripts as a skill leaf.** A new `project-management/pipeline-tools` skill (no trigger phrases; it exists to be installed at a known path) ships bash scripts the agents call by path: `wave-create.sh` (worktrees at BASE plus the base gate, prints BASE and paths), `step-verify.sh <worktree> <base> <step-file>` (runs `acceptance_cmd`, compares to `acceptance_expect`, checks the scope subset and the single commit, prints PASS or FAIL with the evidence), `step-merge.sh` (merge, ledger row, commit, cleanup), `dod-run.sh` (one PASS or FAIL line per Definition of Done item), `find-formatter-gates.sh`, `check-shared-model.sh`. Scripts are testable the way `finance/` already is; prose is not. The supervisor's definition shrinks to policy: what to do with a verdict.
2. **Pull, not push.** Every stage reads its inputs from the artifacts directory; a prompt carries only keys and paths (`plan-name`, `artifacts-dir`, branch, phase or step id). The worker reads its own step file (I1 to I3); the lead developer can pass the planner a

path and heading instead of pasting the milestone section. No Opus stage re-emits text that is already on disk.

3. **Flat nesting.** The decomposer never spawns. Amendments bubble up as `RESULT: needs-amendment` and the supervisor (or lead developer) spawns the planner. The deepest chain becomes lead-developer, supervisor, then worker, decomposer or planner at level 2, one below the default limit of three layers under the main conversation.
4. **Two-tier verification.** Scripts decide PASS or FAIL mechanically; the model reads the verdict and the failure evidence. The one judgment that stays with the model is "check the check" (content manufactured to satisfy a grep), applied to PASS cases whose acceptance is a count or pattern match, by reading the diff of `files_in_scope`.
5. **Model and effort tiers, after measurement.** Once mechanics are scripted the supervisor's remaining work is reading verdicts, choosing retry, revise or escalate, and writing revision notes, which is a Sonnet candidate. The decomposer stays on Opus: its errors cost whole revision cycles. The planner stays on Opus: one call per milestone. Workers get a `tier`: `standard` on Sonnet, `lite` on Haiku for verbatim-apply steps. Per the API guidance, test the capable model at lower effort (Opus at medium for decomposer and supervisor) before any model swap; a cascade also splits the prompt cache by model.
6. **Instrumentation as an artifact.** The ledger's Steps table gains `tokens`, `tool_uses`, `wall_s`, `retries`; the supervisor fills them from the Agent tool's completion notice for each worker (this harness reports subagent tokens, tool uses and duration there) and records its own and the decomposer's totals in the phase notes. A tag `archive/<project-name>-m<n>` (I9) keeps step files and reports after the squash, so post-mortems have both the numbers and the evidence.
7. **A smoke-test fixture.** A small repository with a committed two-phase, four-step plan that runs end to end in minutes. Run it after every Claude Code upgrade and every agent edit; it is the only way the harness facts in E1 get checked before a real run depends on them. The root CLAUDE.md asks for an end-to-end validation but ships no fixture.
8. **One schema bump.** Bundle the step-schema changes (machine-checkable `acceptance`, resolved paths, `tier`, no `rollback`, opt-in gate step) into a single major version of decomposer, supervisor and worker rather than a series of breaking releases.

9. **Shared model as an enforced include.** Start with the drift check (I13). If edits to the section stay frequent, generate the three AGENT.md files from a per-agent source plus one `shared-model.md` with a build script, and keep the generated files committed so installation does not change.
10. **Worker context hygiene.** Subagents load every CLAUDE.md level the main conversation loads, including the user's global one; the sub-agents docs list `omitClaudeMd: true` as the opt-out. Set it on the worker: the packet is meant to be all it knows, and it removes both the shell-rule conflict (E5) and a few thousand tokens from every worker context.

Cost evaluation methods

Measure before changing any pin: no run of the current definitions has produced token, turn or wall-clock numbers. The order is instrument, baseline on fixtures, attribute by stage, then ablate one lever at a time and judge each by cost per completed phase at equal quality.

1. Instrument every run (zero model cost). Three sources exist now:

Source	What it gives	How to use it
Agent tool completion notice	Total tokens, tool uses and duration per subagent	Supervisor and lead-developer copy them into the ledger Steps row and phase notes (A6).
`claude -p --output-format json`	Session `usage`, `total_cost_usd`, cost per model	Per-model cost separates the Opus stages from the Sonnet workers in one number; run the smoke fixture headless to get it.
OpenTelemetry (`claude_code.token.usage` by `type` input, output, cacheRead, cacheCreation; `claude_code.cost.usage`; `agent.name` attribute)	Per-request tokens and cost tagged by agent	Set `OTEL_LOG_TOOL_DETAILS=1` or custom agent names collapse to `custom`; export to any OTLP backend and group by `agent.name`.

Transcripts live under `~/.claude/projects/<project>/` with subagent files beside the session, but their per-message format is undocumented; prefer the two documented channels.

2. Fix a benchmark set. Three plans of known size, re-run from the same starting commit each time: the smoke fixture (two phases, four steps), a mid-size synthetic plan (three phases, about ten steps, with one spec-level step, one verbatim step and one `judgment` step per phase) and one real milestone from a current project, chosen once and frozen. Each run records tokens by stage and model, cost at list price, wall-clock per phase, retries, revisions, amendments, human escalations, and whether every DoD passed. Variance between identical runs sets the noise floor; two or three repeats per configuration are enough to see a 20 percent effect.

3. Attribute cost by stage. From the ledger rows and the per-model cost, compute the share taken by planner, decomposer, supervisor, workers and lead-developer, and within the supervisor the share of turns that are git mechanics versus decisions. The working hypothesis from the protocols is that decomposer and supervisor dominate; the first instrumented run confirms or refutes it before anything is optimized.

4. Ablate one lever at a time. Each row is a configuration run against the benchmark set; keep the change only if cost per completed phase falls and DoD pass rate, revision count and escalation count do not rise.

Experiment	Change	Expected effect	What would reject it
X1	Scripts for wave-create, verify, merge-record (A1)	Supervisor turns per step from about 10 to about 4; supervisor output tokens down by half or more	Any verification miss the prose version would have caught
X2	Workers read their own step file (A2)	Supervisor output tokens down by the step-file size per phase	Worker failures from reading `route` or `depends_on`
X3	Opus at `medium` effort for decomposer and supervisor	Output tokens down 30 to 50 percent at those stages	Revision rate or acceptance mis-specification rate rises

Experiment	Change	Expected effect	What would reject it
X4	Sonnet supervisor after X1	Supervisor cost halved	Wrong retry, revise, escalate decisions; missed check-the-check cases
X5	Haiku `lite` worker for verbatim-apply steps	Those steps at a twentieth of the token price	Retry rate on lite steps above the standard rate
X6	No gate step (I5)	One worker and worktree cycle saved per phase	A phase DoD failure the gate would have reported earlier than the supervisor's own run
X7	Spec-level `actions` by default (F3)	Step files and decomposer output shrink; worker output grows	Revision rate on spec-level steps rises
X8	`omitClaudeMd: true` on the worker (A10)	Worker input tokens down by the CLAUDE.md sizes; fewer deviation notes	A worker missing a build convention it needed
X9	Whole milestone done by one Opus session at `high`, no pipeline	The control: what the pipeline must beat on cost and on DoD pass rate	If the control wins on both, the pipeline earns its keep only on parallel wall-clock or on tasks too large for one context

5. Derived metrics to track per run.

- Cost per completed phase and per merged step, at list price.
- Verification catch rate: steps the supervisor failed after the worker reported pass. A rate near zero with Sonnet workers argues for the Haiku tier; a high rate argues for better step specification, not a stronger worker.
- Parallelism factor: sum of step durations divided by phase wall-clock; near 1.0 means the waves are not running concurrently, which foreground launches may cause and which is undocumented either way.

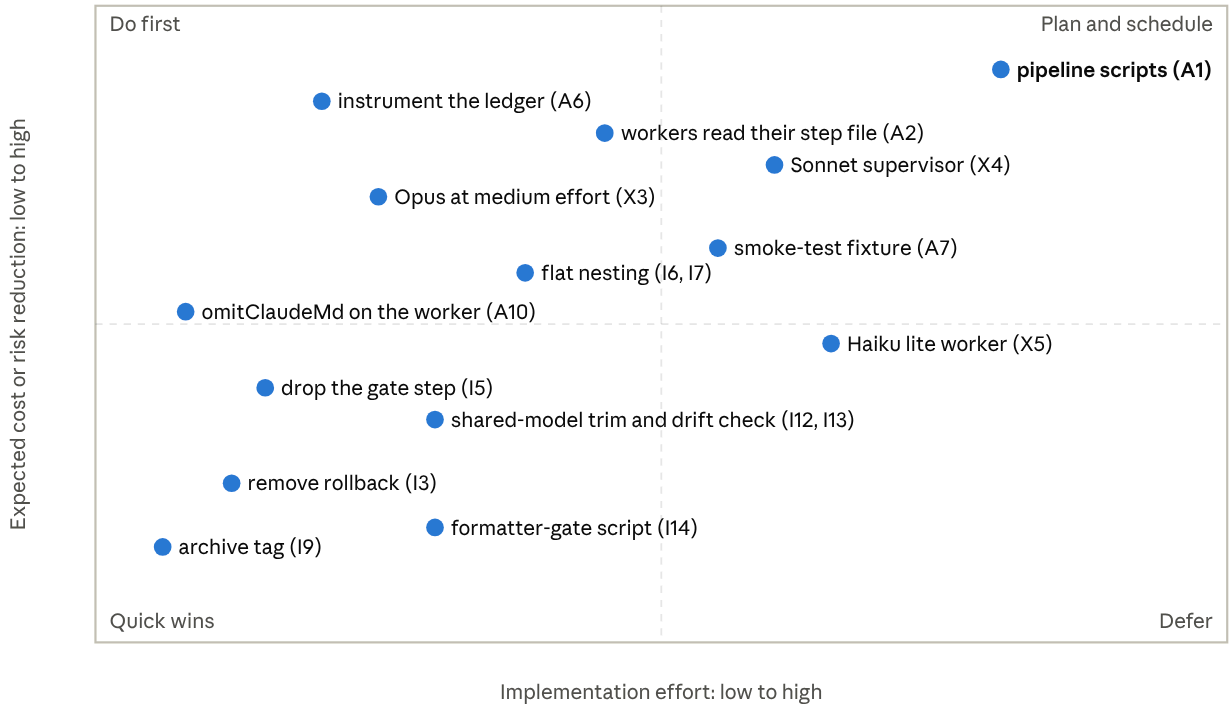
- Cache hit ratio from the `cacheRead` metric; a low ratio means a definition prefix is changing between calls.
- Rework ratio: tokens spent in retries, revisions and amendments over total.

6. Decision rule. Adopt a change when it lowers cost per completed phase on all three benchmarks and changes no quality metric beyond the noise floor. Report every rejected experiment in the ledger of the fixture repository so the next review does not repeat it.

Prioritized recommendations

Do the measurement hooks and the one-commit cuts now, ship the schema and script changes as one major release, and change model or effort pins only on benchmark evidence.

Instrument and the small cuts first; scripts are the large win



recommendations by effort and impact · 14 items

Position is a judgment, not a measurement: the x axis is implementation effort in this repository, the y axis the expected cost or risk reduction.

1. **Now, one commit each, no measurement needed.** Ledger instrumentation columns (A6), `omitClaudeMd: true` on the worker (A10), the archive tag before squash (I9), the shared-model drift check (I13), the judgment-step protocol (I8), README and CLAUDE.md corrections (I15, I16).
2. **Next, as one major release of decomposer, supervisor and worker.** Workers read their step file (I1, I2), resolved paths, `tier` and no `rollback` (I3), machine-checkable `acceptance` (I4), opt-in gate step (I5), flat nesting (I6, I7), the `pipeline-tools` scripts (A1) and the smoke-test fixture (A7) that proves the release end to end.
3. **After the benchmark runs.** Opus at medium effort (X3), a Sonnet supervisor (X4), the Haiku lite worker (X5) and spec-level `actions` by default (X7), each kept only when the decision rule in Cost evaluation methods passes.

What not to change, by design: ground-truth re-verification by the supervisor, projection into self-contained step packets, base assertion without self-repair, disjoint scopes with worktree isolation, and the revision and amendment loops. These are the mechanisms that make a cheap-worker fleet safe to merge from; measure their overhead, do not remove them.